

Agent changes — this session

Every change to the agent since the pipeline baseline (26fa7df, public 0.859), by author, with the before/after and the measured effect. The evaluator, catalog, public labels and API contract were **not** touched.

Score progression

checkpoint	who	public	adversarial (96)	note
pipeline baseline	team	0.859	0.684	dual-track + span rerank + FixedPolicy
+ windowed recommendations	KW	0.891	0.764	superseded by the elimination scan
+ tuned BM25 field weights	corainexia	0.898	0.725	merged onto the scan
+ elimination scan (turn 3)	KW	0.898	0.725	(this is what main carried)
+ facet-agreement rerank signal	corainexia	0.903	0.788	PR #3
+ retrieval-recall fixes	KW	0.903	0.788	this branch; dev 0.909, holdout 0.895
+ query facet extraction	corainexia	—	—	PR #5
+ multi-route anchor retrieval	xiaotong0329	—	—	PR #6
+ category tail match	Elinengu	0.9128	0.7914	change 5; fixes the last public miss — public 200/200
+ learned boilerplate stoplist	Elinengu	0.9128	0.7914	change 6; score-neutral by design
+ negative facet evidence	Elinengu	0.9143	0.7917	penalise contradiction of a stated facet; profile + budget signals measured and rejected — docs/team/rerank_signals.md
+ pair spans + word-bounded matching	Elinengu	0.9159	0.7944	keep key:value associations intact; worst hard bucket +4.7 MRR pts
+ constraint-ledger investigation	Elinengu	0.9159	0.7944	change 9; no code shipped — six ledger operations measured, all flat or worse; corrected a wrong diagnosis in src/rerank.py; two dead functions deleted
+ narrow first slate (4, 10)	Elinengu	0.9199	0.7981	change 10; one config default. Started as a conditional-MMR assessment — MMR measured and rejected, the deferral it stumbled on kept
+ semantic reranking (S6b)	Elinengu	0.9199	0.7981	change 11; built, measured, removed (code on branch semantic-rerank) — cross-encoder loses on every split; oracle reranking ceiling established at +0.043 / +0.084
+ popularity weight 0.02 → 0.4	Elinengu	0.9305	0.8020	change 12; the tie-break regime fix — every split up, a hard-set miss converted; coordinate-ascent argmax measured and <i>not</i> shipped
+ pool-aware clarification wording	KW	0.9305	0.8020	change 13; score-neutral by construction — ask_attribute unchanged, simulator never reads message. Realism for Pillar II / Presentation

Net: **public 0.859 -> 0.9305, adversarial 0.684 -> 0.8020**. 70/70 tests pass. The thirteen core-agent changes are detailed below; supporting tooling and docs follow. Change 9 moved the score by exactly zero and is recorded in full anyway — a measured no-change is the evidence that keeps the shipped design chosen rather than assumed. Change 10 is the same lesson from the other side: the proposal that prompted it was rejected on its own terms, and the win came from understanding *why* it appeared to work.

Dashes mark changes that landed while several branches were merging in parallel, where no clean before/after was captured at the time. Changes 5-8 were measured in isolation instead — toggling the one parameter in a single

process — which is the method `.claude/skills/record-change/SKILL.md` now requires, precisely because differencing across merges attributed one teammate's gain to another's change.

Change 1 — Recommendation strategy: elimination scan (Kwong Weng)

Files: `starter/agent.py`, `tools/sweep.py`

Problem

After ~turn 3 the simulated customer has disclosed all ~4 constraints and the ranking stops changing. The old agent showed the **same top 10 every turn 3-10**, so any target not in that top 10 was a guaranteed miss — 12 of 200 public sessions, and far more on generic-constraint targets.

What changed

Two iterations, both gated by AgentConfig flags so the old behaviour is one switch away:

1. Windowed scan (b71db6d): freeze the reranked pool on the first list

shown, then walk it in windows — turn 3 ranks 1-10, turn 4 ranks 11-20, turn 5 ranks 21-30, ... Re-snapshot on a real new constraint or an intent override. AgentConfig: `scan_windows`.

2. Elimination scan (96d5385, ships): a product shown and not hit on is a

confirmed non-target (the session would have ended). So each turn: drop everything already shown, return the top 10 of the re-ranked **survivors**. No frozen pool, no cursor — `rerank()` runs every turn anyway, so new constraints and overrides are reflected automatically, and a target that drifts down the ranking is still a survivor and surfaces a turn later (no gap).

```
# starter/agent.py _shortlist(), simplified
shown = self._shown.setdefault(sid, set())
if (state.override_turn or 0) != self._shown_override.get(sid, 0):
    shown.clear() # a pre-override list confirms nothing
    self._shown_override[sid] = state.override_turn or 0
picks = [asin for asin, _ in candidates if asin not in shown][:limit]
shown.update(picks)
```

New AgentConfig fields: `elimination_scan = True`, `hold_until_stalled = False`. `first_recommend_turn` stays the start-turn knob; a dev+holdout sweep (`tools/sweep.py: elim1/2/3, elim_hold`) picked **turn 3** — earlier starts freeze a poor MRR on under-informed early hits.

One correctness fix

An intent-override session's pre-override list often already contains the target, but the evaluator ignores hits until the override lands. Without clearing the shown-set on override detection the scan sails past it — this collapsed override to 0.11 in one iteration; the clear restores it to 1.00.

Effect

Public 0.891 (windowing) then elimination about the same on public but **fixes the boundary-MRR regression** (0.756 -> 0.883) and is simpler. Public hit@10 0.940 -> 0.990; only 2 misses left. On the adversarial set the two variants trade (windowing 0.764, elimination 0.725) — elimination re-ranks every turn, so on all-generic constraints the "no preference" junk spans jostle a borderline target away. Change 4 fixes that.

Change 2 — Tuned BM25 field weights (corainexia, PR #2)

File: `src/index.py` commit 201c00d

```
DEFAULT_WEIGHTS (parent_asin, title, categories, features, details, store, description)
before (0.0, 6.0, 4.0, 2.5, 2.5, 1.5, 1.0)
after (0.0, 8.0, 5.0, 6.0, 6.0, 0.5, 0.25)
```

Heavier title / categories / features / details; store and long marketing description almost silenced. The original tuple was inherited from the starter and had never been tuned.

Effect

Public 0.8928 -> **0.8982** (+0.005), almost entirely MRR (0.804 -> 0.821) — hit@10 was already saturated, so this ranks the found target higher. Every scenario's MRR improved; holdout rose more than dev, so it generalises.

Change 3 — Facet-agreement rerank signal (corainexia, PR #3)

File: src/rerank.py commit 2d7f92b

Adds a fourth term to the rerank score. `_facet_agreement()` runs `src/facets.py:extract()` over the customer's accumulated text and over each candidate, and counts matching facet values (material, colour, size, style, use_case):

```
total = ( span_weight * coverage
+ retrieval_weight * (bm25 / max_bm25)
+ popularity_weight * popularity
+ facet_weight * facet_score ) # facet_weight = 0.3 <-- new
```

This is docs/team/ideas.md Idea 2 (partial) — the reranker had been ignoring FacetStore, which is built for every product but read by nothing.

Effect

Public 0.8995 -> **0.9031**; recovers the MRR that Change 4's query filter costs on its own and adds more (public MRR 0.814 -> 0.825).

Change 4 — Retrieval recall: hold declines out of the query (Kwong Weng)

Files: src/state.py, src/rerank.py commit 960eed0

Problem

`retrieve()` queries `state.full_text()` = every utterance joined. From turn 4 the simulator's "*I don't have an additional preference for feature / use_case / style / material / colour / size*" leaks feature, style, material, colour, size, category into the BM25 OR-query and the span matcher — terms that match huge swathes of the catalog and dilute the target. Recall **degrades within a session**: adversarial "target in the 300-pool" fell from 96% (turn 3) to 81% (turn 10).

What changed

- `Utterance.declined` flag, set in `observe()` when `NO_PREFERENCE_CUES`

`matches.full_text()`, `focused_text()` and `query_spans()` skip declined utterances; a decline also no longer counts as a "productive" turn.

- `RerankConfig.depth` **200 -> 300** — rescore the whole retrieval

pool, not a prefix. ~12% of cluster-target sessions had the target in the pool but past rank 200, where the span signal never applied. (No-op on public; helps only the adversarial set.)

Effect — recall (the point of the change)

target in 300-pool at end of session	before	after
public	98%	100%
adversarial	81%	95%
adversarial: degenerate_card / homogeneous_cluster	56% / 81%	88% / 100%
end-of-session pool-rank p90 (public / adversarial)	87 / 279	16 / 200

The within-session degradation is gone. Score: adversarial 0.725 -> **0.788** (hit@10 0.802 -> 0.885); public flat on its own, net **+0.005** once combined with Change 3.

Change 5 — Category tail match in the reranker (Elinengu)

Files: src/rerank.py — commit f322a52. Full write-up: docs/team/category_tail_match.md

Problem

The customer's opening names the target's coarse category, which the evaluator builds from the two most specific levels of the target's category path (coarse_category, evaluator/local_evaluator.py:126): Novelty > Women becomes *"I'm looking for Novelty Women"*.

Ancestor overlap alone cannot exploit that. A deeper candidate — ... > Novelty > Women > Tops & Tees > T-Shirts — shares every ancestor the target has and scores just as well, even though its own two most specific levels (Tops & Tees T-Shirts) are never mentioned in the opening. In the last remaining public-set miss (public_0020) this produced a **159-way tie** in the reranker.

What changed

Score the *tail* of each candidate's category path, not its ancestors: a point for each of the candidate's two most specific levels whose tokens are fully contained in the opening. Matching is by token containment rather than by parsing the opening's template, so paraphrased private-set wording still works. Generic levels such as Clothing, Shoes & Jewelry are excluded.

```
for part in cleaned[-2:]:                # the two most specific levels
    part_tokens = set(terms(part))
    if part_tokens and part_tokens <= opening_terms:
        score += 1.0
```

RerankConfig.tail_weight = 0.8. The 0.6-1.5 range scores identically on dev and holdout, so this sits mid-plateau rather than at either split's argmax.

Effect

Measured in one process by toggling tail_weight between 0.0 and 0.8, everything else held constant.

	off	on	delta
Public set	0.907000	0.912801	+0.0058
Public MRR	0.8273	0.8417	+0.0144
Public MTTC	3.060	2.985	-0.075
Adversarial set	0.786037	0.791375	+0.0053
Adversarial MRR	0.6798	0.6949	+0.0151

The gain is entirely in ranking. Hit rate does not move on either set (public 1.000, adversarial 0.8854) — the tail match reorders candidates retrieval had already found, which is what a reranking signal should do.

Change 6 — Learned boilerplate stoplist (Elinengu)

Files: tools/build_stoplist.py, src/stoplist.py, src/text.py, tests/test_stoplist.py — commit f322a52

Problem

BOILERPLATE in src/text.py was 24 hand-written tokens with no record of what evidence produced any of them. IMPLEMENTATION.md proposed replacing it by dropping the most frequent ~200 catalog terms.

That proposal was wrong. Ranked by document frequency over the 50,000-product catalog, polyester is 72nd, cotton 83rd, black 97th, leather 111th and spandex 141st. Dropping the top 200 deletes every one — and those are

precisely the constraints the customer discloses, because `intent_card()` inserts a material at position 0 and a colour at position 1 of every card. Meanwhile `asin`, a genuine member of the hand-written list, sits at rank 10,379 with 0.0% document frequency. Frequency fails in both directions.

What changed

The usable signal is *where* a token occurs, not how often. Structural metadata lives in the `details` dict and appears almost nowhere else:

structural	in details		attribute	in details
department	100.0%		spandex	1.2%
dimensions	99.6%		cotton	2.5%
manufacturer	99.6%		polyester	3.3%
inches	97.7%		black	13.4%

The gap between 16% and 96% is empty of attribute words, so the threshold is read off the catalog's own distribution rather than tuned against a score. `tools/build_stoplist.py` selects tokens with document frequency $\geq 5\%$ and $\geq 90\%$ of occurrences inside details, and writes `src/stoplist.py`. It reads `data/catalog.jsonl` and never opens `data/public_set.jsonl`, so it cannot fit the public sessions.

The learned list reproduces 14 of the 24 hand-written terms and finds 22 more nobody wrote down — every month name and year 2014-2022, harvested from `Date First Available: August 15, 2019`.

Ten terms stay hand-written, deliberately. `imported`, `machine`, `wash`, `closure`, `care` and friends are care-and-origin phrases in `features/description`. Two statistics were tested and both fail to separate them from real attributes: by document frequency `closure` (38.6%) outranks `polyester` (21.8%); by spread across the 12 largest category buckets `polyester` (CV 0.51) sits between `imported` (0.49) and `wash` (0.60), with `black/white` (0.34/0.31) inside the scaffolding band. What separates them is that a shopper does not choose between "imported" and "not imported" — semantics, not frequency. The evidence sits beside them in `src/text.py` so the experiment is not retried.

Effect

Measured in one process by swapping `src.text.BOILERPLATE` between the two lists.

	hand-written (24)	learned (46)
Public set	0.912801	0.912801
Adversarial set	0.791636	0.791375
Sweep dev / holdout	0.9187 / 0.9029	0.9190 / 0.9029

Score-neutral, as predicted before it was written. Boilerplate removal strips a median of one token from a ten-token query, and `MAX_QUERY_TERMS = 60` never binds — 0 of 200 sessions come close. The adversarial -0.0003 is one session's reciprocal rank; hit rate and MTTC are unchanged on both sets.

It ships for auditability and for robustness on a catalog nobody has hand-inspected, not for points. `tests/test_stoplist.py` holds the invariants that keep a future regeneration honest.

Change 7 — Negative facet evidence in the reranker (Elinengu)

Files: `src/rerank.py`, `src/facets.py`, `src/policy.py`, `tools/sweep.py`, `tests/test_components.py` — full record with the two rejected sibling signals in `docs/team/rerank_signals.md`

Problem

Every rerank term is positive evidence: span coverage, facet agreement, category and tail alignment. None can separate a candidate that *satisfies* "color: grey" from one that merely *doesn't contradict* it — a black-only shirt matches "cotton shirt" spans exactly as well as a grey one and loses nothing for being black. The customer's constraint rules products out; the scorer only ever ruled products in.

What changed

`_facet_conflicts()` counts stated facet values the candidate contradicts, and the score subtracts `facet_conflict_weight (0.4) * conflicts`. A conflict requires all three of:

1. the customer stated a value for the attribute (`extract_query_facets`);
2. the candidate resolves that attribute too — **silence is never punished**,

missing data is not disagreement;

3. the stated value (and every alias — grey/gray is the vocabulary's only

synonym pair) appears nowhere in the candidate's text as a word-bounded substring. This guards against `extract()`'s first-match-wins: a "black/grey reversible" belt extracts `color: black` but still contains "grey" and must not be penalised.

The override fix the first version needed. Judged against `full_text()`, the penalty punished the target for *obeying an intent override*: after "actually, ignore black — I need grey", the stale "black" was still extracted first and grey-only products were demoted for contradicting it — the adversarial `generic_override` bucket dropped 0.673 -> 0.626 MRR. Conflicts are therefore judged against `extract_query_facets(state.focused_text())` — the currently authoritative turns, identical to `full_text()` until an override fires — which restored the bucket to exactly 0.673. Positive terms still read `full_text()` deliberately: stale positive evidence mildly boosts wrong candidates, stale negative evidence actively demotes the right one.

Weight 0.4 sits at the low end of the dev plateau (0.9224 at 0.4, 0.9226 at 0.8): a penalty term gets the smallest weight that works, and at 0.4 one conflict can reorder saturated ties but cannot overturn a genuine span lead (each matched span is worth ≥ 1.12).

Effect

Measured in one process by toggling `facet_conflict_weight` between 0.0 and 0.4:

	off	on
Public set	0.912801	0.914287 (hit stays 200/200)
Sweep dev / holdout	0.9207 / 0.9010	0.9224 / 0.9021
Adversarial set	0.791375	0.791697 — no bucket regresses

The signal was designed for the `homogeneous_cluster` bucket, and that hypothesis was wrong: bucketmates share the stated facets by construction, so there is nothing to contradict inside the cluster (+0.002 only). The gain is cross-cluster precision on the general distribution. Five unit tests hold the guards (multi-value products, silence, override staleness, weight-0 no-op).

Two sibling signals were measured and **not** shipped, per the remove-dead-options convention: profile-weighted facet agreement (dev +0.0013 does not reproduce on holdout, -0.0008 — the customer already discloses their profiled preferences verbatim, so the profile is redundant with the transcript) and budget/price closeness (rejected before implementation: 0 of 200 public intent cards contain a budget, 0.45% of the catalog can produce one). `docs/team/rerank_signals.md` carries both measurements.

Change 8 — Pair spans and word-bounded span matching (Elinengu)

Files: `src/text.py`, `src/state.py`, `src/rerank.py`, `tools/sweep.py`, `tests/test_components.py` — full record in `docs/team/rerank_signals.md` §2

Problem

Spotted by reading the `public_0020` transcript. The customer's disclosure "Heather Grey: 90% Cotton, 10% Polyester" is a mapping — colour-variant → composition — but `constraint_spans()` splits on `[. ; : , \n]`, severing "heather grey" from *its* "90 cotton 10 polyester". A candidate that pairs the composition differently (an 80/20 heather grey) matches all the fragments exactly as well as the target. Fragments ask "mentions 90% cotton at all?"; the message's evidence is

"says it *about heather grey*?"

Second, latent bug: coverage tested span in text unanchored, so "90 cotton" also matched "190 cotton" (1-4 catalog products per numeric span).

What changed

`pair_spans()` (`src/text.py`): splits only on sentence separators (`. ; \n, " - "`), keeping colon/comma-joined *key:value* content together; strips the leading simulator framing; minimum 3 words. Catalog copy repeats these blocks verbatim, so the joined form is still an exact substring of the target: `df("heather grey 90 cotton 10 polyester") = 511` products vs 612 for "heather grey" alone. `query_pair_spans()` (`src/state.py`): same turn-1/declined exclusions as `query_spans()`, and drops anything already emitted as a fragment so no evidence is counted twice. `* rerank()` adds `pair_weight (0.8) x matched_pairs`, flat 1.0 per pair — the pair's value is the intact association, not its length. Both fragment and pair matching are now word-bounded (the product text is token-joined, so padding with single spaces anchors spans at token edges).

What was tried first and rejected: the obvious diagnosis — eight fragments from one line over-weight that line — led to three de-weighting prototypes: per-utterance groups as sum over $\sqrt{k}$ (public 0.9139), mean (0.9134), best-single-span (0.9025). All flat or worse; the 8-of-8-vs-5-of-8 fragment gradient is load-bearing. The fix is adding the lost association, not weakening the fragments.

Effect

Measured in one process, one toggle at a time:

	baseline	+ word-bounded	+ pair spans (0.8)
Public set	0.914287	0.914937	0.915887 (hit stays 200/200)
Adversarial set	0.791697	0.791697	0.794375
Sweep dev / holdout	0.9224 / 0.9021	0.9223 / 0.9040	0.9233 / 0.9048

The hard-set gain sits entirely in `homogeneous_cluster` (MRR 0.431 -> **0.478**) and `generic_override` (0.673 -> 0.680); no other bucket moves, no hit is lost. This is the bucket Change 7 could not touch: bucketmates share every fragment by construction, but they do not pair the compositions the same way — the intact association is the discriminator that survives saturation. `public_0020` itself goes from rank 4 to rank 1. Weight 0.4-1.5 scores identically on dev and holdout; 0.8 sits mid-plateau. Six new unit tests.

Change 9 — Constraint-ledger investigation: measured, not shipped (Elinengu)

Files: `src/rerank.py` (comments), `src/state.py`, `src/index.py` (dead code), `docs/team/rerank_signals.md`, `docs/team/signal_descriptions.md`, `docs/team/hard_cases.md`, `IMPLEMENTATION.md`

Problem

A proposal to replace utterance-replay in `DialogState` with an explicit ledger of typed `Constraint` records — slot, value, turn, polarity, status — updated by `CARRY / UPDATE / ADD / DELETE / DONTCARE / NEGATE`, with open-ended slots held multi-valued so "Water Resistant" cannot displace "machine washable".

Two open items in our own docs pointed the same way: "fix or delete the override weight" (`hard_cases.md`) and "per-constraint provenance / partial overrides" (`IMPLEMENTATION.md` §3).

What changed

Every operation was measured against the evaluator before any of it was built, and **none of it shipped**. The findings:

- **The override is never a retraction.** `behavior_for()` draws `old_value` and

`new_value` from the *same target's* intent card. Across all 46 override sessions in the two eval sets, not one replaces an exclusive facet value with a different one — 25/30 public are cross-slot ("Buckle closure" → "leather"), 4/30 are feature → feature, and the single material → material case repeats the same value. `UPDATE` has no case to fire

on, which also closes both open items above: there is nothing for `PRE_OVERRIDE_WEIGHT` to express.

- `**focused_text()` in the conflict path is a turn-1 filter, not a staleness

guard. **The comment in `src/rerank.py` claimed stale post-override values punished the target; single-value extraction over full history picks a contradicted value in 0 of 30** sessions.** The real mechanism is that `coarse_category()` emits category levels drawn from the same vocabulary as the `style/use_case` facets, so "I'm looking for Pants Casual" extracts `style=casual`. Variants A and B below are bit-identical, which proves it.

- **Two dead functions deleted:** `DialogState.query_terms()` and

`CatalogIndex.search_phrases()`, neither with any caller.

Effect

Turn-1 exclusion (A/B/C) and multi-valued facets, all four splits:

variant	dev	holdout	public	hard
shipped baseline	0.9233	0.9048	0.9159	0.7944
A: -turn1, keep focused	0.9226	0.9035	0.9150	0.7944
B: -turn1, full history	0.9226	0.9035	0.9150	0.7944
C: full history, +turn1	0.9233	0.9048	0.9159	0.7920
multi-value agreement	0.9225	0.9045	0.9153	0.7921
multi-value conflict only	0.9233	0.9048	0.9159	0.7949
multi-value both	0.9225	0.9045	0.9153	0.7919

Net effect of what shipped:

	before	after
Public set	0.915887	0.915887
Adversarial set	0.794375	0.794375
Tests	57/57	57/57

Exactly zero. The deletions are score-neutral by construction and the rest is comments and documentation. What it buys: a wrong explanation removed from the shipped code before it misled the next person, two open items closed as not-worth-doing rather than left inviting a rebuild, and four negative results recorded with numbers. Turn-1 exclusion is the one worth remembering — it removes every false conflict against the target (8 public, 5 hard) and *still* loses score, because measuring only the harm and never the benefit is how a plausible fix hides.

Change 10 — Narrow the first slate (Elinengu)

Files: `starter/agent.py` (one default), `tools/sweep.py`, `tests/test_components.py`, `docs/team/ideas.md`, `IMPLEMENTATION.md`

Problem

The investigation started as an assessment of a proposed **conditional MMR** diversity term: spread the shown slate across plausible interpretations so the target is more likely to appear *somewhere* in the top 10, trading rank for Hit@10 because Hit@10 carries 0.50 of the score and MRR only 0.30.

The premise is no longer true of this agent. Public-set Hit@10 has been **1.000** since change 5. There is no coverage left to buy, and MRR is precisely what a diversity penalty spends.

MMR was implemented and measured anyway. Across 260 public slates it moved the target *into* a slate it was otherwise outside of **zero** times, and pushed it *out* of one 21 times (hard set: 2 in, 9 out). Hit@10 moved on no set at any lambda. The reason is structural: disclosed constraints are copied verbatim from the target's own metadata, so the head of the ranking is a cluster built around the target's attributes — on the ambiguous slates the gate selects, the target's similarity to the head (0.417) is *higher* than the head's own internal similarity (0.394). The penalty lands hardest on the target.

It nonetheless scored slightly up (public 0.9159 → 0.9176), and tracing why is what produced this change: ejecting the target from turn 3's slate leaves it a survivor for turn 4, when the next disclosed constraint has re-ranked it higher. The gain was **deferral, not diversity** — the \$S7 marginal-value trade obtained by accident. `list_size_ramp` buys deferral directly.

What changed

One default in `starter/agent.py`:

```
list_size_ramp: tuple[int, ...] = (4, 10) # was (10,)
```

Four candidates on turn 3, ten from turn 4. `_shortlist` already indexed the ramp correctly; no other code changed. The elimination scan is what makes the deferral free — the six held back are the top of the survivor list next turn, so the same walk is paced in finer steps rather than truncated.

Effect

	before	after
Public set	0.915887	0.919892
Adversarial set	0.794375	0.798056
dev / holdout	0.9233 / 0.9048	0.9268 / 0.9096
generated (200)	0.9181	0.9197
Tests	62/62	62/62

The first-slate plateau, measured in one process across four sets:

ramp	dev	holdout	generated	hard
(10,)	0.9233	0.9048	0.9181	0.7944
(3, 10)	0.9254	0.9146	0.9212	0.7968
(4, 10)	0.9268	0.9096	0.9197	0.7981
(5, 10)	0.9295	0.9100	0.9210	0.8001
(5, 5, 10)	0.9272	0.9044	0.9187	0.7934
(5, 10) + MMR	0.9284	0.9076	0.9173	0.7994

Sizes 3, 4 and 5 all beat the flat ramp on all four sets, so (4, 10) ships as the midpoint. (5, 10) has the better mean and wins dev and hard outright, but choosing it after seeing the table is the argmax-fitting the house rule exists to prevent; the decision rule was fixed before (4, 10) was measured.

The whole gain is MRR bought with MTTC, at the ~13:1 odds \$3 prices. On public, MRR 0.8513 → 0.8690 (×0.30 = +0.0053) against MTTC 2.975 → 3.040 (efficiency −0.0065, ×0.20 = −0.0013), netting the +0.0040 observed.

What did not move: Hit@10 on public (1.000) and hard (0.885) is unchanged — this change buys rank, not coverage. And the last row is the one that closes the original question: MMR layered *on top of* the ramp is worse on all four sets, so once deferral is supplied properly the diversity term is pure cost. It is recorded as rejected in `docs/team/ideas.md` Idea 3.

Costs, stated plainly. Generated-set Hit@10 slips 0.995 → 0.990 — one session that now runs out of turns. (5, 5, 10) is the informative negative: holding narrow a *second* turn regresses holdout and hard below the flat floor, because each session carries four constraints disclosed at up to two per turn, so by turn 4-5 no further evidence is arriving. The win is one narrow turn, not a direction to push further. Every delta here also sits under the ±0.02 holdout noise floor the skill specifies; the evidence is that they are positive on four independent sets at once, not that any single split is decisive.

Change 11 — Neural cross-encoder reranking, built, measured, removed (Elinengu)

> **Removed from this branch.** The stage never ran on the scored path — off by > default, and a no-op without the gitignored weights — so it was deleted rather > than carried as unused dependency-bearing code. Full working state preserved on > the semantic-rerank branch. The measurements below stand; they produced the > oracle ceiling that change 12 then exploited.

Files (on branch semantic-rerank): src/semantic.py, tools/fetch_model.py, requirements.txt, docs/team/semantic_rerank_setup.md, plus wiring in starter/agent.py, tools/sweep.py, tests/test_components.py, README.md, ARCHITECTURE.md, .gitignore

Problem

Every rerank signal is lexical, and docs/competition_specification.md lists semantic reranking as an innovation direction. Before building anything, an **oracle** reranker (target forced to rank 1 whenever it is in the pool) fixed the ceiling for any reranking work at all:

	dev (120)	holdout (80)	public (200)	hard (96)	generated
baseline	0.9268	0.9096	0.9199	0.7981	0.9197
oracle	0.9638	0.9620	0.9631	0.8823	0.9590
gap	+0.037	+0.052	+0.043	+0.084	+0.039

The proposed ambiguity gate was measured before implementation too. At the suggested thresholds (tied_leaders >= 8) it fires on **73%** of public sessions — an always-on stage, not a fallback — though it does discriminate (mean RR 0.774 when firing against 0.987 when quiet). Shipped thresholds are tighter and fire on 28% of rerank calls.

What changed

src/semantic.py (S6b) reranks ambiguous clusters with cross-encoder/ms-marco-MiniLM-L6-v2 and fuses by RRF, not score addition — cross-encoder logits are uncalibrated and one matched span is worth ~1.12 on the symbolic scale. Runtime is onnxruntime + tokenizers over a **23.2 MB** int8 graph: upstream publishes ONNX exports, so there is no torch, transformers or sentence-transformers dependency and no export step. Weights are gitignored; tools/fetch_model.py downloads them.

Disabled by default, and every failure path is a no-op.

Effect

variant	dev (120)	hard (96)
off (shipped)	0.9268	0.7981
on, weight 0.7	0.9211	0.7944
on, weight 0.3	0.9249	0.7959
on, depth 20	0.9236	0.7940

	before	after
Public set	0.919892	0.919892
Adversarial set	0.798056	0.798056
Tests	62/62	68/68

Zero on the scored path, and negative everywhere the stage is on. The weight column is the finding: 0.7 → 0.3 → 0 recovers baseline monotonically, and an optimum at zero means the signal carries no usable information. The mechanism is that on the 162 fired turns with the target in the rescored head, fusion moved it **up 46 and down 74** (mean rank 7.63 → 8.77) — anti-correlated, not merely miscalibrated. Domain mismatch is the likely cause: MS MARCO pairs a natural-language question with prose, while here the query is simulator boilerplate and the document a token-joined catalog blob.

Cost: mean turn 30.7 ms → 389.8 ms, p95 73.7 → 1347.8 ms, max 1.48 s.

Two things did *not* move, and both were verified rather than asserted. The scored path is bit-identical — checked by stashing the whole branch and re-running both sets, comparing every session's rank and turn, not just the totals. And

the degradation path was exercised: with the stage **on** and the runtime uninstalled, results are identical at no latency cost.

Three implementation notes, all self-caught

- The proposal's "protect strong lexical evidence" guard was built as a `+1.0`

bonus on the fused score. RRF scores here top out near 0.028, so that constant did not protect, it *promoted* — hoisting a unique-span holder from symbolic rank 40 to rank 1. Rewritten as a rank clamp. It changed nothing (0.9211/0.7944 either way), because the guard fires on **0 of 8750** candidates examined: inside a pool retrieved by those very spans, no span is unique. Deleted rather than kept as an inert flag.

- The first ablation grid was abandoned mid-run once the bonus bug was found,

rather than reporting numbers that measured a mistake.

- Importing `onnxruntime` and then exiting the interpreter aborts intermittently

on macOS (`recursive_mutex lock failed, SIGABRT`) — enough to make the test suite flaky at exit code 134 while still reporting 68/68 OK. The first version imported the runtime before checking for weights. Reordered so the weights are checked first, which means the no-weights path — the scored agent and the whole test suite — never imports `onnxruntime` at all. Six consecutive clean runs after.

Change 12 — Popularity weight 0.02 → 0.4: the tie-break regime fix (Elinengu)

Files: `src/rerank.py` (one default + comments), `tools/fit_weights.py` (new), `tools/sweep.py`, `tests/test_components.py`, `docs/team/rerank_signals.md` §10

Problem

Dissecting every near-miss session (target rank 2-10 behind a rank-1 impostor; 33 public, 15 hard) showed that **every lexical signal is exactly tied 33/33** — the remaining headroom is a pure tie-break regime. The tie was broken by the retrieval score, which picks the impostor **33/33** (BM25 length normalization favours thin listings: 126 vs 195 tokens on identical matched evidence), while popularity picks the target **31/33** (the target is a real purchase, hence a reviewed product) but was weighted 0.02 against retrieval's 1.0 — right 94% of the time, drowned 50:1. The same table killed three candidate signals before implementation: title boost (impostor 11:6), match density (impostor 27:33), span contiguity (tied).

What changed

One default: `RerankConfig.popularity_weight` 0.02 → 0.4.

The route there matters as much as the destination. `tools/fit_weights.py` (new, `stdlib` coordinate ascent per Metzler & Croft 2007, fitting the seven non-definitional weights directly on the technical score, **dev split only**) found an argmax at pop 0.8 / retrieval 0.1 / conflict 0. The sealed holdout **confirmed the direction** (+0.019) — not dev overfit — but the argmax regresses the adversarial set 0.7981 → 0.7824, whose targets are deliberately thin and unreviewed. Under the pre-declared rule (holdout keeps gains, hard ≥ baseline, smallest departure wins) the one-weight change is the only qualifier. The argmax stays reproducible as the `weights_argmax` sweep row.

Effect

	before	after
Public set (official)	0.919892	0.930502
Public MRR	0.869	0.901
Adversarial set (official)	0.798056	0.801978
Adversarial hit	0.885	0.896 (a converted miss)
dev / holdout	0.9268 / 0.9096	0.9418 / 0.9136
Tests	68/68	70/70

Every split up, public hit 200/200 kept, no public scenario regresses (boundary MRR 0.704 → 0.86). Plateau: popularity 0.1 / 0.3 / 0.4 / 0.5 are all $\geq$ baseline on all four splits — 0.4 is mid-plateau with both neighbours measured. New tests pin all eight shipped weights and demonstrate the tie-break flip. This implements the "learn the rerank weights" idea the repo carried in four places, with the twist that the honest deliverable was knowing *when not to trust the fit*.

Change 13 — Pool-aware clarification wording (Kwong Weng)

Files: src/phrasing.py (new), src/facets.py (shared helper), src/policy.py (call the helper), starter/agent.py (AgentConfig + _respond), tools/sweep.py, tests/test_components.py

Problem

FixedPolicy sets ask_attribute="other" every turn — the score-optimal choice (other returns two constraints of any type and never whiffs; a specific attribute can return zero and then retires itself). But policy.question() is a fixed table, so the customer hears "Is there anything else that matters for this one?" on turns 1, 2, 3, 4 ... Pillar II asks for "structured, proactive clarification prompts that guide user convergence"; a demo of the same sentence five times is the opposite.

Two subagents (explore_profile_policy, explore_profile_prefilter) confirmed ask_attribute cannot productively change on this evaluator — so the fix is the English only.

What changed

src/phrasing.py:clarify() builds the message. ask_attribute is untouched. Once the customer has disclosed something and recommendations have started, it takes the live reranked pool and picks the facet among material / colour / style / size / use_case (not asked, not declined) that the pool is most evenly split on — the same gain_ratio InfoGainPolicy uses — and names its top 2-3 values in one of three rotated templates:

```
"For the material, I'm seeing leather and canvas - do you have a preference?"
"For how you'll use it, the pool is split across work, everyday and party. Does one matter more to you?"
```

No facet qualifies, or turn 1 → a four-way rotation of the broad question so no sentence repeats. The whole body is wrapped so a phrasing bug degrades to the broad question, never an empty turn. InfoGainPolicy._distributions and the phrasing layer now share one helper, facets.weighted_value_counts.

Follow-up — relaxed gating (same commit series)

The first cut gated the grounded path on state.productive_turns $\geq$ 1 and state.turn_count $\geq$ first_recommend_turn. Inspecting public_0198 (the latest-hitting public session, turn 9) showed all nine of its clarifications were the broad fallback: its disclosures are single words ("leather", "black", "PU") that never form a multi-word constraint span, so productive_turns stayed 0 the whole session and the turn gate never opened until turn 3 anyway.

Relaxed to: grounded from turn_count $\geq$ 2 (after the opening line the retrieval pool reflects something the shopper said), no productive_turns requirement, and the per-facet split thresholds loosened (_MIN_COVERAGE 0.35→0.25, _MAX_TOP_SHARE 0.85→0.90). Among qualifying facets the voiced one now rotates by turn_count % count instead of always the single most-split, so a session on the grounded path varies the facet each turn. Result on the public set: the grounded path fires on 98% of turn- $\geq$ 2 clarifications and in 199 of 200 sessions (was ~0). Score unchanged — still zero by construction.

Effect

	before	after
Public set	0.930502	0.930502
Adversarial set	0.801978	0.801978
dev / holdout	0.9418 / 0.9136	0.9418 / 0.9136
Tests	64/64	70/70

Exactly zero, by construction — measured in one process (tools/sweep.py rows natural_off / natural_on), per-scenario components identical. The simulator never reads message. The change buys demo / Presentation /

Innovation realism, and the shared helper removes a duplicated loop. Default on;
 AgentConfig(natural_questions=False) restores the fixed strings byte-for-byte. Implements the "Question phrasing from the candidates" idea listed under S4.

Example (real runs, natural_questions on)

```
public_0198 [the latest-hitting public session - all broad before the follow-up]
T1 -> "To point you in the right direction: anything else you'd want me to factor in?"
T2 -> "...there's a mix of leather and canvas here - for the material, does one stand out?"
T3 -> "...for sizing, I'm seeing small, adjustable and large - do you have a preference?"
T4 -> "...style-wise, the pool is split across classic, casual and elegant. Does one matter more to you?"
T5 -> "...there's a mix of work, outdoor and travel here - for how you'll use it, does one stand out?"
...                                     -> HIT T9

public_0002 [browsing]
T2 -> "To narrow this down: there's a mix of casual, classic and elegant here - style-wise, does one stand out?"
T3 -> "...on colour, I'm seeing black, brown and gold - do you have a preference?"
T4 -> "...for how you'll use it, the pool is split across work, everyday and party. Does one matter more to you?"
```

Before, every one of those turns was "Is there anything else that matters for this one?".

Supporting work (Kwong Weng)

file	what
tools/hard_cases.py	Adversarial session generator + per-bucket scorer. Scans the frozen catalog, buckets every product by an adversarial property, samples 16 each. -- run scores the agent grouped by bucket.
data/hard_set.jsonl	96 generated sessions (6 buckets: homogeneous_cluster, budget_only_signal, boilerplate_soft, degenerate_card, generic_override, cross_category_collision). Public-set schema; scored by the unmodified evaluator.
tools/sweep.py	build_configs() — added plain, elim1/2/3, elim_hold1/2 for the start-turn sweep.
docs/team/ideas.md / ideas.pdf	Reranking & recommendation-strategy ideas, each with the measured result: elimination scan (1a/1b), decline filter (1c), facet / category / MMR / learned-weights (2-6).
docs/team/hard_cases.md / .pdf	Failure analysis of the adversarial set and the prioritised fix plan.
agent_summary.pdf	Rewritten (c7757af) for the current elimination-scan workflow: the loop, one turn stage-by-stage, recommendation timing, and a round-by-round table per scenario.
examples.pdf	Two annotated agent<->simulator transcripts (one hit, one miss).

Also merged this session (teammates, non-agent)

commit	author	what
a13d8b5	Elinengu	Customer_Simulator_Explainer.pdf
80bd4f8	Elinengu	tools/observe.py, tests/test_observe.py — session observer / monitor
0f085ec	Elinengu	docs/team/intent_override_retrieval.{md, pdf}
01ffdcdb	Elinengu	docs/team/reranking_explained.{md, pdf}
(this change)	Elinengu	docs/team/category_tail_match.{md, pdf} — the last public-set miss, and the no-overfit evidence

Current agent settings (after all changes)

stage	knob	value	file
S1 index	FTS5 field weights (title, cat, features, details, store, desc)	8, 5, 6, 6, 0.5, 0.25	src/index.py

stage	knob	value	file
S5 retrieval	pool size / RRF k / focused-route weight	300 / 60 / 0.8	src/retrieval.py
S6 rerank	span / retrieval / popularity / facet weight	1.0 / 1.0 / 0.02 / 0.3	src/rerank.py
S6 rerank	category (ancestor) / tail weight	0.4 / 0.8	src/rerank.py
S6 rerank	facet-conflict penalty (vs post-override facets)	0.4	src/rerank.py
S6 rerank	pair-span weight (key:value associations, word-bounded)	0.8	src/rerank.py
S6 rerank	length bonus / depth	0.12 / 300	src/rerank.py
S3 state	pre-override utterance weight	0.35 (inert, and correctly so — change 9)	src/state.py
S3 state	declined utterances held out of every retrieval view	—	src/state.py
S4 policy	FixedPolicy: other, then feature-ladder	—	src/policy.py
S7 timing	first_recommend_turn / confidence margin / earliest	3 / 0.20 / 2	starter/agent.py
S7 timing	elimination_scan / hold_until_stalled	on / off	starter/agent.py

Not touched (organizer-owned)

evaluator/local_evaluator.py, data/catalog.jsonl, data/public_set.jsonl, and the five frozen files at the root of docs/: agent_api_contract.json, baseline_results.json, competition_specification.md, evaluation_config.json, submission_rules.md.

Everything under docs/team/ is ours — see docs/README.md for the split.